Lambda Expressions

TP2

Samir Genaim

What they are?

```
Arrays.sort(a, new Comparator<A>() {
            public int compare(A x, A y) {...body...}
        }));
```

```
Arrays.sort(a, (A x ,A y) -> {...body...} );
```

Intuitively, when ever you need an anonymous (and thus local) class that implements an interface with only one abstract method (we call it a functional interface), you can replace it by an expression of the following form that corresponds to the implementation of that method

```
(Type1 arg1, Type2 arg2,...) -> { ...body... }
```

This form is called lambda-expressions. They exist since Java 8.

They are not anonymous classes (I)

1. In practice they are not anonymous classes, but for simplicity you can consider them as if they were instances of objects that implement that interface
2. The main difference from anonymous classes is in that they cannot have a state

```
foo(a1,a2, new SomeFunctionalInterface() {
        int count = 0;
        public int f(int x, int y) {
            count++;
            return x+y+count;
        });
```

Anonymous class with a state. Cannot be replaced by a λ expression

If the field count was defined in the class that includes the call to foo, then it is possible because λ-expressions can access anything that is defined in the creation-context, exactly like anonymous classes.

They are not anonymous classes (II)

- ★ We have seen that anonymous classes exists only at the level of Java, but not the Java virtual machine. The compiler will actually generate auxiliary classes for them.

- ★ Lambda expression are supported at the level of the Java virtual machine. They are more lightweight than anonymous classes.

- ★ Use them instead of anonymous classes whenever you can!

- ★ Btw, the name "lambda" comes From functional programming. In that context, functions are typically called lambda-expression ...

Matching a Functional Interface?

When using a λ-expression instead of an object that implements an interface T, the compiler ask the following questions to understand to which method we refer:

1. Does T have only a single abstract (unimplemented) method? T can have more static methods or methods with default implementation.
2. Does the parameters of the λ-expression match the parameters of that abstract method (of point 1)?
3. Does the return type of the λ-expression match the return type of that abstract method?

If the answer to these questions is YES, then that abstract method is the one that the λ-expression refers to, otherwise the compiler issues an error.

Some syntactic sugar in Java (I)

We can drop the types of the arguments, the compiler will infer them automatically (if it cannot infer them it will complain):

```
(int x, int y) -> { ... }
(x,y) -> { ... }
```

If it has only 1 parameter we can drop the parenthesis:

```
(x) -> { ... }
 x -> { ... }
```

Some syntactic sugar in Java (II)

If the body has a single (not a return) instruction we can drop the curly brackets:

```
(x,y) -> { System.out.println(x+y); }
(x,y) -> System.out.println(x+y)
```

If the body has a single return instruction we can drop the curly brackets and the keyword return:

```
(x,y) -> { return x>y; }
(x,y) -> x>y
```

A function instead of λ-expression

```
public interface MyPrinter{
    public void print(String s);
}


MyPrinter myPrinter1 = C::foo;

MyPrinter myPrinter2 = o::foo;
```

foo is a static method of the class C and it matches print of MyPrinter, i.e., has the same arguments (and types) and return type

foo is an instance method of the object o and it matches print of MyPrinter, i.e., has the same arguments (and types) and return type

A function instead of λ-expression

```
public interface MyPrinter{
    public void print(String s);
}

MyPrinter myPrinter1 = (s) -> { System.out.println(s); };

MyPrinter myPrinter2 = System.out::println;
```



It seems like there is a support for this syntax at the level of Java virtual machine, so it might save defining an extra method in the .class (by the compiler) as done for normal lambda expressions ...

More resources

The following is a good tutorial on lambda expressions:

http://tutorials.jenkov.com/java/lambda-expressions.html